

I think that is important to discuss some modifications I have made to this project's dataset before discussing the EDA and feature engineering. After some reflecting, I had come to the conclusion that my current dataset might not have enough records for the machine learning models I desired to use. I proposed my plan to ChatGPT, and it agreed that the project would benefit from an increase in records for models to train on. After collecting more data from basketball reference the updated dataset now contains 11 NBA seasons from 2015 to 2025 which increased the number of records up to 5,968 which is around five times more than the original which only had around 1,100 total records. The other minor alteration I made within the csv file was to change the format of the column SEASON from '2025-26' to '2025'. I made this decision not only because it is simpler but also because it allowed me to now operate on the column within the EDA and feature engineering steps.

Following the guide provided on Canvas, the first thing that was worked on was the missing values present within the dataset. In order to count or effectively evaluate the number and percentage of missing values per column, I decided to use a function I had learned within a previous graduate course.

```
# check missing value in each column.
def missingValueCount(df):
    columns=df.columns
    total_count=df.count()
    dict={}
    i=1
    for c in columns:
        missing_value_count=df.filter(col(c).isNull()).count()
        ratio=round(missing_value_count/total_count, 3)
        dict[c]="Total missing value: "+str(missing_value_count)+ "; % of Missing Value: "+str(ratio)
    return dict
```

Above is an image that shows the original function I learned within Pyspark. All I need to do was to slightly modify this function so that would work properly within google colab. When looking at the missing value counts, what stood immediate was that a large majority of the columns did not have any missing values. An outlier column in terms of number of missing values was the 'Awards' column with around 90% of values missing. This large number of missing values justified my decision to drop the column from the dataset. The rest of the 9 columns with missing values were only missing around 1 – 7%. The normal conclusion in this circumstance is to just remove the rows that have these missing values. However, this solution does not work for this project because our goal is to evaluate season by season so removing any player data would significantly impact the experiment. When looking closer into the columns that had these missing values, a common pattern between all these columns emerged. Using FG% (Field Goal %) as the example, this column is one that is calculated by taking the players FG (Field Goals Made) and dividing it by FGA (Field Goals Attempted) to get their FG%. So when looking at rows where FG% was null, I found that the number of FG and FGA were both equal to 0. This basically means that this player's field goal percentage was not calculated because this player never took a single shot. All the remaining columns '3P%', '2P%', 'eFG%',

'FT%', 'TS%', '3PA', 'FTr', and 'TOV' were all similar to what I described for 'FG%'. Since this was occurring within all the other columns with missing values as well, it led me to the conclusion that I should fill the null values with 0.

Before I can discuss the data imbalance and the strategy I plan on using to solve, I first needed to create my target variable using the data I had now cleaned. The plan for the target variable was to utilize the 'PTS' (Points Per Game) and 'WS' (Win Shares) columns to classify if a player did or did not have a breakout season. This would be done by taking the difference between the current season's values and season after values. In order to get these current and future values for PTS and WS in the same row, I asked ChatGPT for some help. It explained that I should use '.shift(-1)' as this would allow me to create a column for future values by referencing the actual value present in the next row. The only downside using shift was that it created rows with null values for these future value columns. I decided to check the number of unique values for players before dropping these rows and after dropping the ones with null values and found that around 400 unique players were removed. I used AI to help generate code to confirm if these removed players only played for 1 total season within the NBA and my hypothesis was correct with the number of unique players removed matching exactly with those who only played for 1 year. I then needed to calculate the differential between the current and future seasons to determine the amount of improvement for players. I decided to use percentage change for the PTS and actual differential for WS. With these two new calculated columns I was finally about to create my 'breakout' column using the criteria of an increase of 20% for PTS and 2 total WS. If the player meets these requirements, they would be labeled as 1 and if they failed to meet both requirements they would be labeled 0.

I utilized `value_counts` to see how the data was split for my target variable and found it was imbalanced with a 93/7 split. After evaluating the options, I decided to go with the strategy of class weighting to address the imbalance. Through research and discussion with ChatGPT it seemed like this might be the ideal approach since a majority of the classification models I was planning on using have class weight built into the model. This makes me believe that it is simple to implement and will involve modifying the original data, which I believe to be ideal. However, through feedback or after running the models if the outcomes are not desirable, I am open to adjusting this approach.

Next up is the discussion around the feature engineering that was done. The first decision I made was to drop 7 features from the dataset. 'Rk' showed the order that each player appears when sorted alphabetically, which has no meaningful value to an NBA player's ability to breakout. 'Team' represents the 3 initials of the NBA team(s) the player played for each season. This column as well does not have any meaning or relation to a player's statistics or ability to breakout. 'MPTotal' showed the total number of minutes the NBA player played in each season. This column represents the same information as MPavg which is just the average minute played per game, and I decided that it would be best to remove MPTotal since it was redundant information. I am going to talk about the last 4 features together since they are the fields I created to calculate

the breakout target variable. I believed it was important to remove these columns since they are directly correlated with the target variable and will likely be predicted as the top performing features for predicting breakout since they directly influenced the creation of the target variable. The last column I want to discuss is 'Pos' or Position. I decided to use the `get_dummies` code provided in class to one hot encode this categorical variable in 0's and 1's. The last part of this step was to calculate feature importance. Since I had columns that contained negative values, I was unable to utilize Chi2 which led me to choose Anova F-test as my alternative to evaluating feature importance. models that I should consider.

In this final paragraph, I will be discussing the impact of the preprocessing done as well as why I utilized box plots to check for outliers. Before the preprocessing, the dataset had a total of 6,200 missing values across its columns with an imbalanced ratio of 93/7 with its target variable. After all of the preprocessing steps have been completed, the dataset has 0% missing values across all columns and since I have gone with a model focused strategy for addressing the target variable imbalance, it has not been handled yet. I decided to use box plots to evaluate and examine the outliers within the most important features. Not only do box plots incorporate IQR within them, but I also believe it is better to visual the number of outliers through box plots compared to some of the more numerical calculation methods.